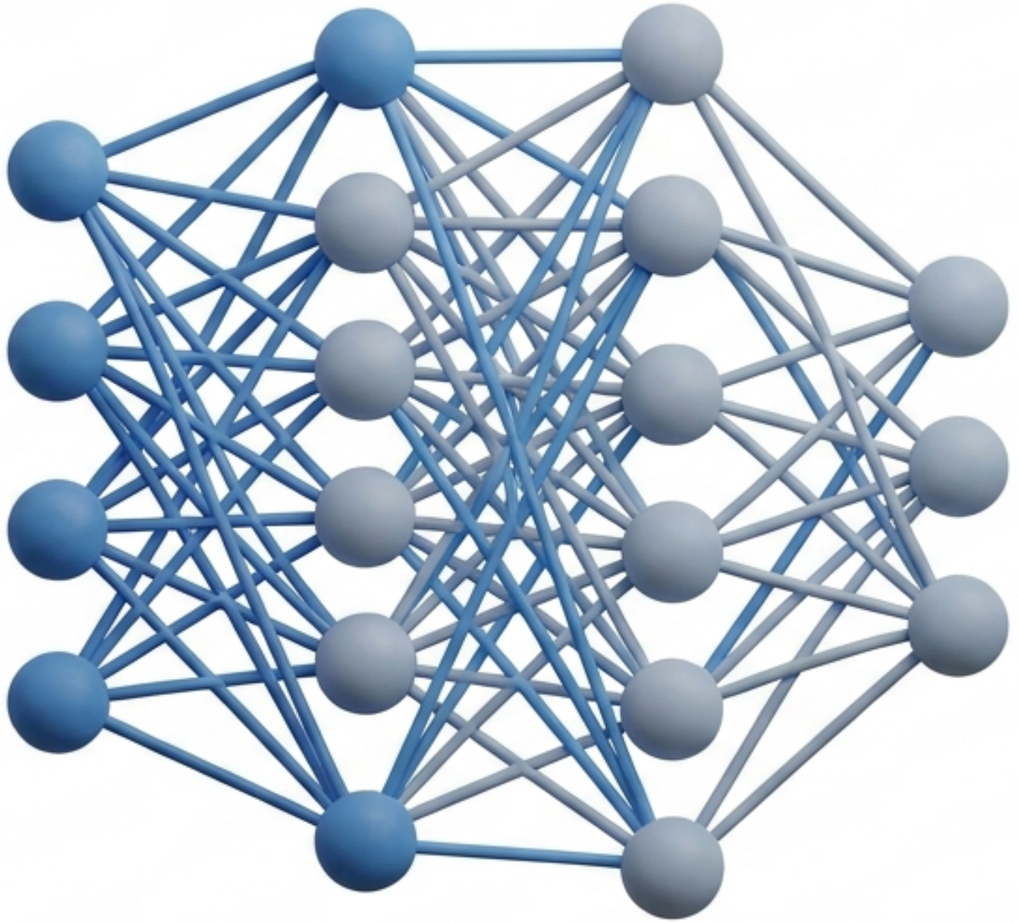
Chương 11: luyện lyện Mạng Nơ-ron Sâu



Giảng viên: [Tên Giảng Viên]

Đại Học Đông Á

Ngày 11 tháng 8 năm 2026

Gemini Notebook

Nội dung Chương 11

1. Vấn Đề Gradient Biến Mất / Bùng Nổ
2. Các Hàm Kích Hoạt Nâng Cao
3. Batch Normalization và Gradient Clipping
4. Tái Sử Dụng Các Lớp (Transfer Learning)
5. Các Trình Tối Ưu Hóa Tốc Độ Học (Fast Optimizers)
6. Lịch Trình Tốc Độ Học (Learning Rate Scheduling)
7. Kỹ Thuật Chính Quy Hóa (Regularization)

Gemini Notebook

Vấn đề Vanishing và Exploding Gradients

Thuật toán **backpropagation** cập nhật trọng số bằng cách truyền đạo hàm (gradient) từ lớp đầu ra ngược về lớp đầu vào.

Vanishing Gradients



Càng truyền ngược về các lớp sâu hơn (lower layers), **gradient** thường càng trở nên **nhỏ dần**.

Hậu quả: Trọng số của các lớp dưới cùng không thay đổi, mạng nơ-ron sâu không thể hội tụ.

Exploding Gradients



Gradient có thể **tăng lên** theo **cấp số nhân**.

Hậu quả: Trọng số lớn đến mức tràn bộ nhớ (thường gặp trong mạng RNN).

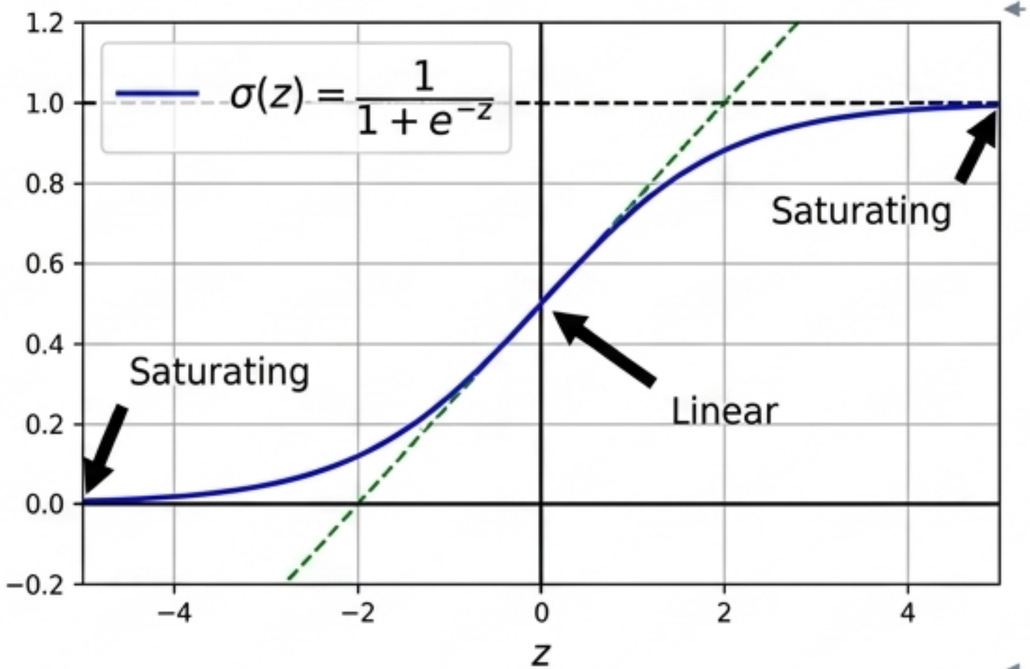
Gemini Notebook

Thủ phạm: Sự Bão Hòa Của Hàm Kích Hoạt Sigmoid

Năm 2010, Glorot và Bengio phát hiện thủ phạm chính: **hàm kích hoạt Sigmoid + khởi tạo trọng số ngẫu nhiên** ban đầu.

Nguyên nhân

Phương sai đầu ra > Phương sai đầu vào. Các giá trị ở lớp trên cùng trở nên rất lớn (hoặc rất âm).



Cơ chế bão hòa

Khi giá trị tuyệt đối lớn, đạo hàm của hàm Sigmoid tiến về. Trọng số ngừng cập nhật.

Hình 11-1: Hàm kích hoạt sigmoid bão hòa ở hai đầu

Gemini Notebook

Giải pháp: Khởi tạo Trọng số Glorot / He (Initialization)



Nguyên tắc cốt lõi: Để tín hiệu chảy bình thường (xuôi & ngược), phương sai của đầu ra và đầu vào ở mỗi lớp phải bằng nhau.

Xavier Glorot

Đề xuất khởi tạo trọng số ngẫu nhiên (phân phối chuẩn) với phương sai:

$$\sigma^2 = \frac{1}{fan_avg}$$

Kaiming He

Đối với hàm ReLU, đề xuất khởi tạo He initialization:

$$\sigma^2 = \frac{2}{fan_in}$$

Gemini Notebook

Thực hành: Code Khởi Tạo Trọng Số trong Keras

Ghi chú: Mặc định, Keras dùng khởi tạo Glorot (phân phối đều).

```python
import tensorflow as tf
from tensorflow import keras

# Sụ dung He initialization (thuơng ket hợp voi ReLU)
dense = keras.layers.Dense(50, activation="relu",
                           kernel_initializer="he_normal")

# Hoac sụ dung phuong sai do LeCun de xuat (ket hợp voi SELU)
dense_lecun = keras.layers.Dense(50, activation="selu",
                                 kernel_initializer="lecun_normal")
```



Gemini Notebook

Điểm mù của ReLU: Vấn đề "Dying ReLU"

Dù hàm ReLU (`Rectified Linear Unit`) là tiêu chuẩn vàng, nó có một vấn đề nghiêm trọng gọi là **Dying ReLU**.



Nguyên nhân

Trọng số cập nhật sao cho tổng tĩnh $z < 0$.

Cơ chế

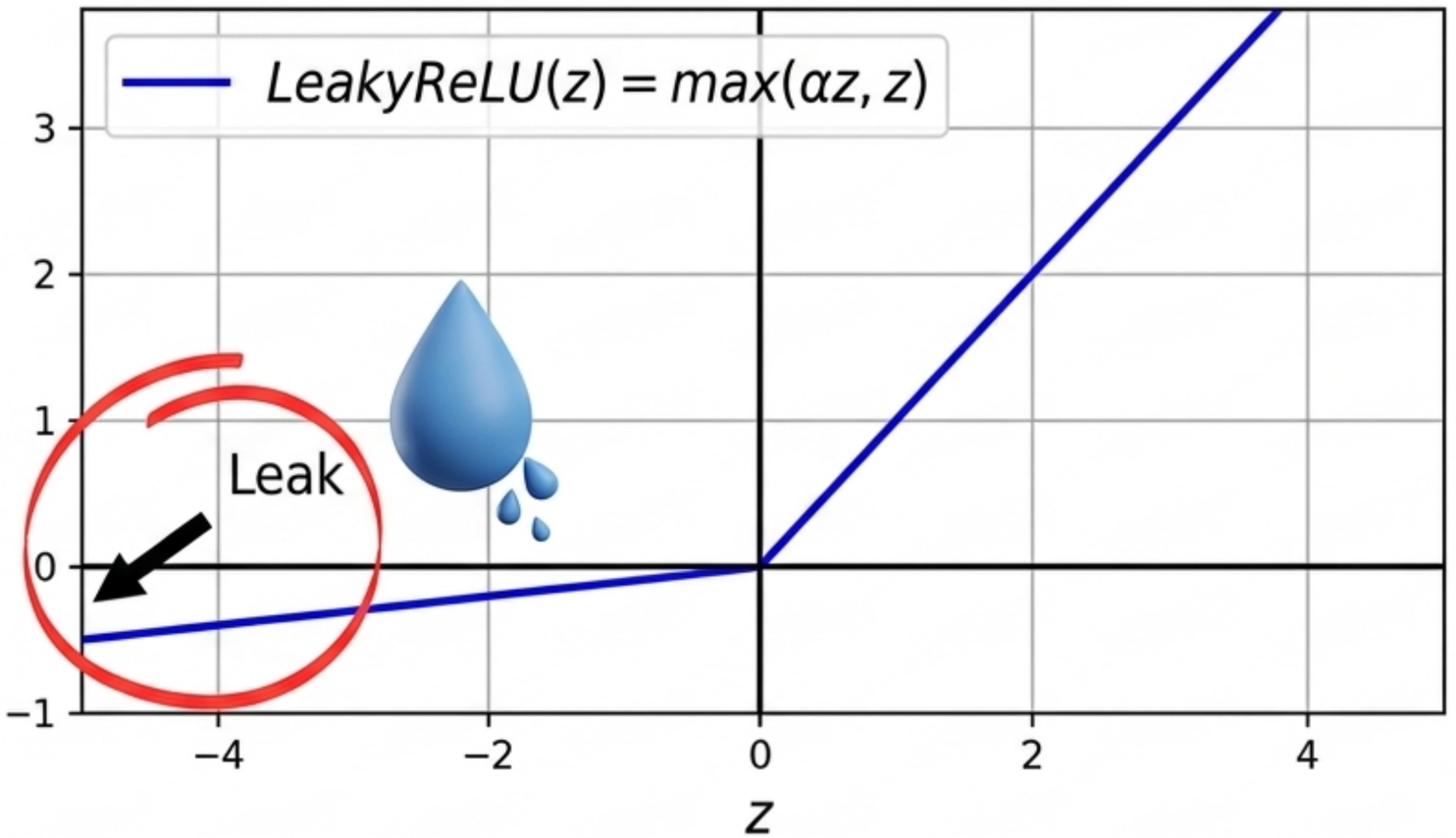
Hàm luôn trả về 0. Đồng thời, đạo hàm khi $z < 0$ cũng bằng 0.

Hậu quả

Nơ-ron "chết" vĩnh viễn và không bao giờ học lại được nữa.

Gemini Notebook

Khắc phục: Hàm Leaky ReLU



Hình 11-2: Hàm kích hoạt Leaky ReLU

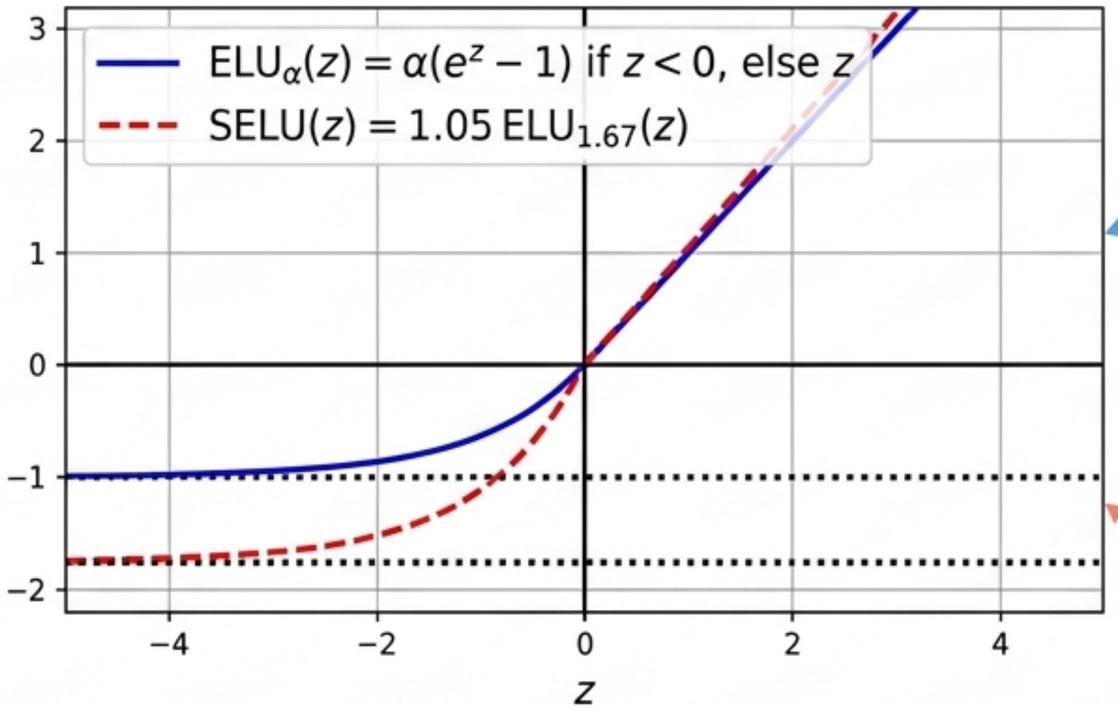
Công thức:
$LeakyReLU_\alpha(z) = \max(\alpha z, z)$

Giải thích tham số: α (thường là 0.01) xác định độ dốc của hàm với các giá trị âm.

Tác dụng: "Độ rò rỉ" (leak) này giúp giữ nơ-ron không bị chết hoàn toàn, tạo cơ hội cho gradient được truyền ngược trở lại.

Gemini Notebook

Biến thể Nâng cao: ELU và SELU



Hình 11-3: Các hàm kích hoạt ELU và SELU

ELU (Exponential Linear Unit)

- Mượt mà ở khắp mọi nơi (đạo hàm liên tục tại z = 0).
- Cải thiện tốc độ học tập.

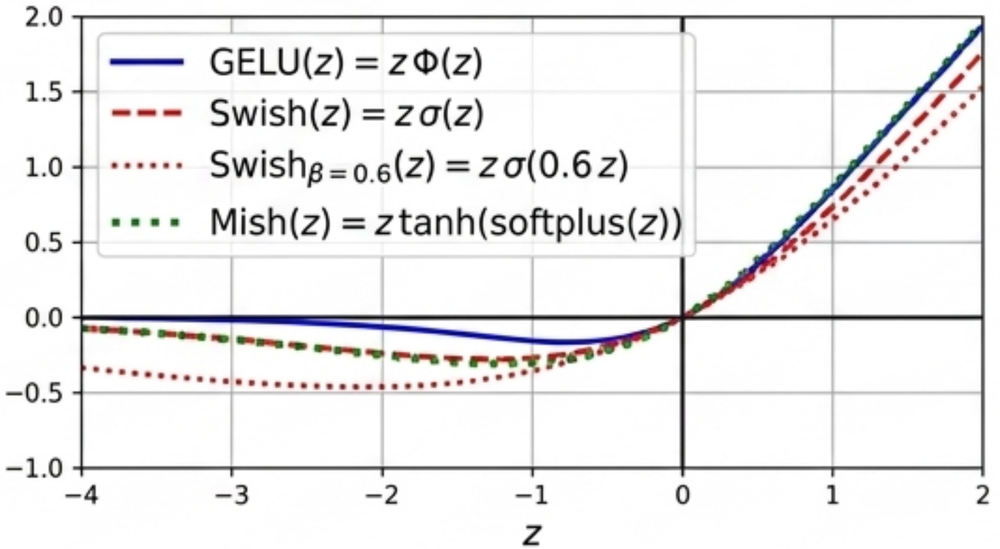
SELU (Scaled ELU)

- Nếu tất cả các lớp dày đặc sử dụng SELU, mạng sẽ tự chuẩn hóa (self-normalize).
- Giải quyết vấn đề vanishing gradient triệt để.

Gemini Notebook

Các Hàm Hiện Đại: GELU, Swish, và Mish

Đặc điểm chung: Đây là các hàm phức tạp, mượt mà và không đơn điệu (non-monotonic).



Hình 11-4: Các hàm kích hoạt GELU, Swish, và Mish

GELU: Vượt trội trong các mạng NLP (như BERT).

Swish: Do Google Brain đề xuất, tối ưu hơn trong các bài toán Image Recognition sâu.

Mish: Một hàm mới với đồ thị gần tương tự nhưng có công thức khác.

Gemini Notebook

Chuẩn hóa Hàng loạt (Batch Normalization)

Bối cảnh: Khởi tạo He và ELU có thể giảm thiểu Vanishing Gradient ở đầu khóa huấn luyện, nhưng không đảm bảo không xuất hiện trong quá trình.

Giải pháp: Sergey Ioffe và Christian Szegedy (2015) giới thiệu Batch Normalization (BN).



1. Tính **trung bình** và **độ lệch chuẩn** của đầu vào tại mỗi lô (**mini-batch**) độc lập.

2. Áp dụng **thay đổi tỷ lệ (scale)** γ

3. Áp dụng **độ dịch chuyển (shift)** β được học qua backpropagation.

Gemini Notebook

Thực hành: Thêm Batch Normalization trong Keras

Kinh nghiệm thực tế: Lớp BN thường được thêm trước hoặc sau Hàm kích hoạt. Đặt nó **TRƯỚC** hàm kích hoạt sẽ hoạt động tốt nhất.

```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(300, use_bias=False), # khong can bias vi BN co thuoc tinh dich chuyen(beta)
    keras.layers.BatchNormalization(),
    keras.layers.Activation("relu"),
    keras.layers.Dense(100, use_bias=False),
    keras.layers.BatchNormalization(),
    keras.layers.Activation("relu"),
    keras.layers. Dense(10, activation= "softmax")
 ])
```

Không cần bias khi sử dụng BN vì BN đã có tham số dịch chuyển (beta)

Vị trí đặt các lớp Batch Normalization: Thường trước lớp Dense hoặc Activation để ổn định phân phối.

Gemini Notebook

Hoạt động của BN lúc Inference (Dự đoán)

Lúc Huấn luyện	Lúc Dự đoán (Inference)
 **Thách thức:** BN tính mean, variance của một mini-batch.	 **Thách thức:** Dữ liệu đến từng cái một, làm sao tính mean và variance?



Giải pháp (Keras xử lý tự động)

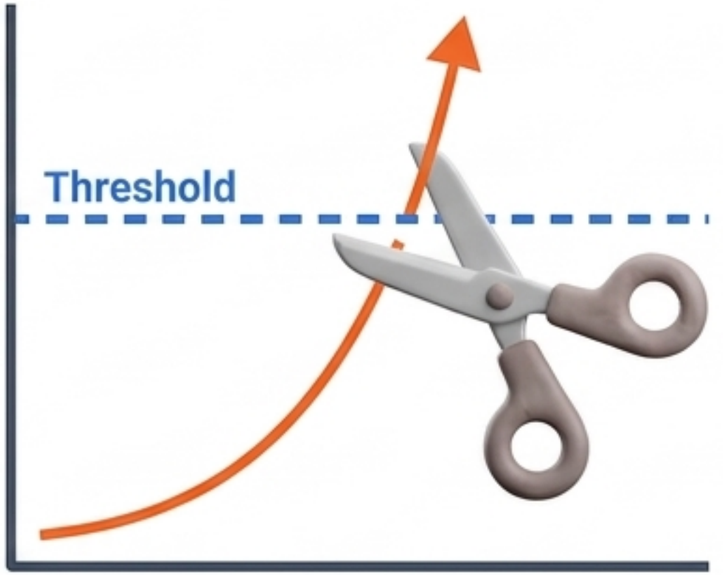
- BN theo dõi moving average (trung bình động) của mean và variance trên toàn bộ tập huấn luyện.
- Keras sẽ tự động sử dụng giá trị này khi bạn gọi `model.predict()`.

Gemini Notebook

Kìm hãm sự bùng nổ: Gradient Clipping

Khái niệm: Kỹ thuật phổ biến để giảm bớt vấn đề Exploding Gradients. Thường được dùng mạnh mẽ trong Mạng Nơ-ron Hồi quy (RNN).

Cơ chế: Cắt (clip) các gradient để chúng không bao giờ vượt quá một ngưỡng (Threshold).




```
# Cat gradient theo gia tri tuyet doi (-1.0 den 1.0)
optimizer = keras.optimizers.SGD(clipvalue=1.0)
model.compile(loss="mse", optimizer=optimizer)

# Hoac cat theo chuan (Norm) de khong thay doi huong cua vector
optimizer = keras.optimizers.SGD(clipnorm=1.0)
```

Gemini Notebook

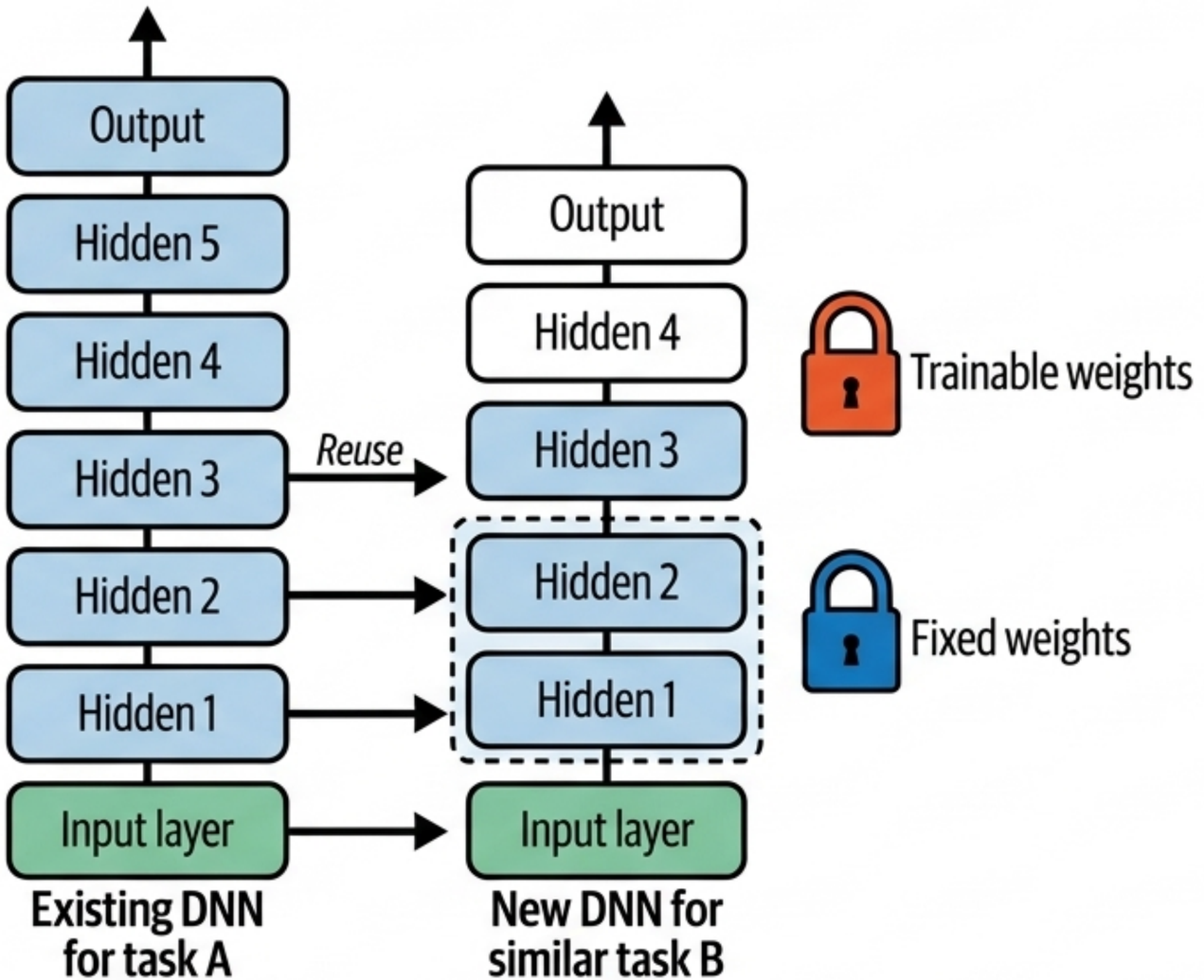
Chiến lược: Học Chuyển Giao (Transfer Learning)

Vấn đề: Sẽ rất lãng phí nếu đào tạo mạng CNN siêu lớn từ con số 0.

Chiến lược "Tái sử dụng":

1. Tìm một Mạng Nơ-ron hiện có thực hiện nhiệm vụ tương tự.
2. Giữ lại các lớp dưới (Lower layers) / Đóng băng (Freeze).
3. Chỉ thay đổi lớp trên cùng (Output layers).

Kết luận: Tái sử dụng các lớp giúp tăng tốc độ huấn luyện đáng kể.



Hình 11-5: *Tái sử dụng các lớp đã được huấn luyện trước*

Gemini Notebook